S07 Object Pooling

CRE132 worksheet

What you'll make

The player shoots a stream of fireballs that burst on the training dummies and walls. Instead of making a new fireball for every shot and destroying it afterwards, you'll make a few at the start and reuse them over and over: an **object pool**. A counter in the top left shows how few objects that needs.

By the end you'll know why games pool things that appear and disappear often, how to write a pool with a `List`, and why a reused object must be reset each time, with `OnEnable` as its "Start".

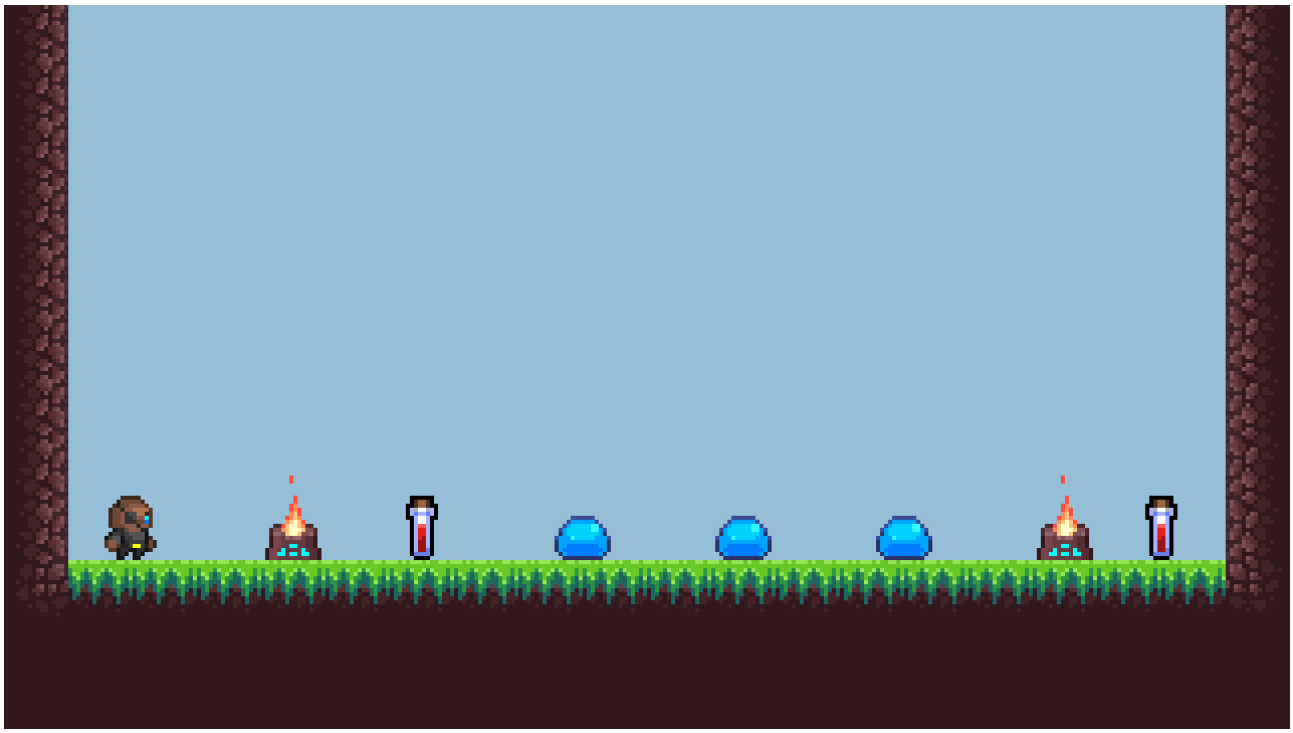


Figure 1: The finished scene: the player, two fires, two potions and three training dummies.

Before you start

1. Unzip `CRE132_S07_ObjectPooling_Starter.zip`. In Unity Hub, choose **Add > Add project from disk** and pick the `CRE132_S07_ObjectPooling_Starter` folder. Open it with Unity **6000.3.11f1**.
2. Open the scene `Assets/_game/Scenes/Shared/S07_ObjectPooling`.
3. Press **Play**. You can walk, run and jump, the fire hurts and the potions heal, but you can't attack, and the counter in the top left never changes. Press Play again to stop.

The scene has two empty objects, **Fireball Pool** and **Impact Pool**, and a **Pool Counter** text on the **HUD Canvas**. The two prefabs are in `Assets/_game/Prefabs/Projectiles`: **Fireball** (a sprite with an Animator, a **Kinematic** Rigidbody 2D and a trigger Circle Collider 2D) and **Fireball Impact** (a sprite with an Animator). The Training Dummies now have 10 health.

Making a script: in the Project window, go to the folder, right-click and choose **Create > Scripting > MonoBehaviour Script**. Type the name straight away and press Enter. The name must match the class name in the code exactly, capital letters included.

1. The pool: ObjectPool

The problem: calling `Instantiate` and `Destroy` for every bullet takes time, and every destroyed object leaves memory for the **garbage collector** to clean up. When it does, the game can freeze for a moment. **The idea:** make some copies at the start, switched off. To fire, find one that's off, move it and switch it on. To "destroy" it, switch it off again.

Make a folder `Assets/_game/Scripts/Utilities` and create `ObjectPool` in it:

```
// A simple object pool: makes some copies of a prefab up front and reuses them
// instead of calling Instantiate and Destroy every time. An inactive object in the
// list is free to use; to give an object back, the object just turns itself off
// (SetActive(false)).
using System.Collections.Generic;
using UnityEngine;

public class ObjectPool : MonoBehaviour
{
    [SerializeField] private GameObject prefab;
    [SerializeField] private int startSize = 10;

    private List<GameObject> pool = new List<GameObject>();

    // For the on-screen counter: how many copies exist, and how many are in use.
    public int TotalCount
    {
        get { return pool.Count; }
    }

    public int ActiveCount
    {
        get
        {
            int count = 0;
            foreach (GameObject pooledObject in pool)
            {
                if (pooledObject.activeSelf)
                {
                    count++;
                }
            }
            return count;
        }
    }

    private void Awake()
    {
        for (int i = 0; i < startSize; i++)
        {
            CreateObject();
        }
    }

    // Hands out a free object, switched on at the given position.
    public GameObject Get(Vector3 position)
```

```
    {
        foreach (GameObject pooledObject in pool)
        {
            if (!pooledObject.activeSelf)
            {
                pooledObject.transform.position = position;
                pooledObject.SetActive(true);
                return pooledObject;
            }
        }

        // Every object is in use, so the pool grows by one. If this happens a lot,
        // raise startSize.
        GameObject newObject = CreateObject();
        newObject.transform.position = position;
        newObject.SetActive(true);
        return newObject;
    }

    private GameObject CreateObject()
    {
        // Made as a child of the pool, so the Hierarchy stays tidy, and switched off
        // until needed.
        GameObject newObject = Instantiate(prefab, transform);
        newObject.SetActive(false);
        pool.Add(newObject);
        return newObject;
    }
}
```

What's going on:

- **`List<GameObject>`** is a list that can grow, unlike an array. `using System.Collections.Generic;` makes it available.
- **`Awake`** fills the pool with `startSize` copies, each made as a child of the pool and switched off straight away.
- **`Get`** looks for a copy that's switched off (`activeSelf` is false), moves it and switches it on. If every copy is busy, it makes one more, so the game never runs out.
- **No "give back" method:** an object returns to the pool just by switching itself off with `SetActive(false)`. Then `Get` sees it as free again.

Add it: add **Object Pool** to **Fireball Pool**, drag the **Fireball** prefab into **Prefab** and leave **Start Size** at 10. Add it to **Impact Pool** too, with the **Fireball Impact** prefab and **Start Size** 5. The same script makes both pools.

Test it: press Play and open **Fireball Pool** in the Hierarchy: ten greyed-out (inactive) **Fireball(Clone)** children.

2. The fireball: Projectile

Make a folder `Assets/_game/Scripts/S07_ObjectPooling` and create `Projectile` in it:

```
// A fireball that flies sideways, hurts anything with Health, shows an impact effect
// and goes back to its pool.
using UnityEngine;

public class Projectile : MonoBehaviour
{
```

```
    [SerializeField] private float speed = 12f;
    [SerializeField] private float lifetime = 1.5f;
    [SerializeField] private int damage = 1;

    private Rigidbody2D body;
    private SpriteRenderer spriteRenderer;
    private ObjectPool impactPool;
    private float timeLeft;

    private void Awake()
    {
        body = GetComponent<Rigidbody2D>();
        spriteRenderer = GetComponent<SpriteRenderer>();
    }

    // Called by the shooter each time the fireball comes out of the pool. Everything
    // that changes per shot is reset here, because a reused object still has the
    // values from its last flight.
    public void Launch(float direction, ObjectPool newImpactPool)
    {
        impactPool = newImpactPool;
        timeLeft = lifetime;
        spriteRenderer.flipX = direction < 0;
        body.linearVelocity = new Vector2(direction * speed, 0f);
    }

    private void Update()
    {
        // Missed everything: give it back to the pool instead of destroying it.
        timeLeft = timeLeft - Time.deltaTime;
        if (timeLeft <= 0f)
        {
            gameObject.SetActive(false);
        }
    }

    private void OnTriggerEnter2D(Collider2D other)
    {
        // Fly through the player who fired it and through other triggers (coins,
        // fire, potions).
        if (other.isTrigger || other.CompareTag("Player"))
        {
            return;
        }

        Health health = other.GetComponent<Health>();
        if (health != null)
        {
            health.TakeDamage(damage);
        }

        GameObject impact = impactPool.Get(transform.position);
        impact.GetComponent<SpriteRenderer>().flipX = spriteRenderer.flipX;

        gameObject.SetActive(false);
    }
}
```

- **`Launch` resets everything** for each shot: the timer, the direction and the speed. This is the big gotcha with pools: a reused fireball still has the values from its last flight.

- **`gameObject.SetActive(false)`** instead of `Destroy`: when the time runs out or it hits something, the fireball goes back to its pool.
- **`other.isTrigger`** lets the fireball fly through the fire, the potions and the player. It only bursts on solid things: dummies and walls.
- **The impact comes from the other pool**, at the fireball's position, facing the same way.

Add it: open the **Fireball** prefab (double-click it) and add **Projectile** to it. Leave **Speed** 12, **Lifetime** 1.5 and **Damage** 1. Save the prefab.

3. The impact: ReturnAfterTime

Create `ReturnAfterTime` in the `S07_ObjectPooling` folder:

```
// Turns this object off after a short time, which gives it back to its pool. Used
// for effects like impacts.
using UnityEngine;

public class ReturnAfterTime : MonoBehaviour
{
    [SerializeField] private float lifetime = 0.3f;

    private float timeLeft;

    // OnEnable runs every time the pool switches the object on, so the timer restarts
    // for each use.
    private void OnEnable()
    {
        timeLeft = lifetime;
    }

    private void Update()
    {
        timeLeft = timeLeft - Time.deltaTime;
        if (timeLeft <= 0f)
        {
            gameObject.SetActive(false);
        }
    }
}
```

- **`OnEnable` is a pooled object's "Start".** `Start` only runs the first time; `OnEnable` runs every time the pool switches the object on, so the timer restarts for each use. The Animator restarts its clip on enable too.

Add it to the **Fireball Impact** prefab, with **Lifetime** 0.3. Save the prefab.

4. Shooting: PlayerShooter

Create `PlayerShooter` in the `S07_ObjectPooling` folder:

```
// Hold Attack to fire fireballs the way the player faces. Fireballs come from a pool
// instead of Instantiate.
using UnityEngine;
using UnityEngine.InputSystem;

public class PlayerShooter : MonoBehaviour
{
    [SerializeField] private InputActionReference attackAction;
    [SerializeField] private ObjectPool fireballPool;
```

```
    [SerializeField] private ObjectPool impactPool;
    [SerializeField] private float timeBetweenShots = 0.12f;
    [SerializeField] private Vector2 firePointOffset = new Vector2(0.6f, 0.55f);

    private SpriteRenderer spriteRenderer;
    private float nextShotTime;

    private void Awake()
    {
        spriteRenderer = GetComponent<SpriteRenderer>();
    }

    private void OnEnable()
    {
        attackAction.action.Enable();
    }

    private void OnDisable()
    {
        attackAction.action.Disable();
    }

    private void Update()
    {
        // IsPressed (held) rather than WasPressedThisFrame, so holding the button
        // keeps firing.
        if (attackAction.action.IsPressed() && Time.time >= nextShotTime)
        {
            Shoot();
            nextShotTime = Time.time + timeBetweenShots;
        }
    }

    private void Shoot()
    {
        float direction = 1f;
        if (spriteRenderer.flipX)
        {
            direction = -1f;
        }

        Vector3 firePoint = transform.position +
            new Vector3(firePointOffset.x * direction, firePointOffset.y, 0f);
        GameObject fireball = fireballPool.Get(firePoint);
        fireball.GetComponent<Projectile>().Launch(direction, impactPool);
    }
}
```

- **`IsPressed()`** is true for as long as the button is held, so holding it keeps firing. `nextShotTime` makes a cooldown, like S01's fire.
- **`fireballPool.Get(firePoint)`** replaces `Instantiate`, then `Launch` sends the fireball on its way and tells it which pool to take impacts from.
- **`firePointOffset`** starts the fireball in front of the player at about hand height, flipped when facing left.

Add it to the **Player**. Set **Attack Action** to **Player/Attack** (J, the left mouse button or the gamepad's East button). Drag **Fireball Pool** into **Fireball Pool** and **Impact Pool** into **Impact Pool**. Leave **Time Between Shots** 0.12 and **Fire Point Offset** 0.6, 0.55.

Test it: hold **J**. Fireballs stream out the way you face and burst on dummies (ten hits each) and walls. Open **Fireball Pool** in the Hierarchy while firing: the same children switch on and off.

5. Counting: PoolCounter

Counting is only there to prove the pool works, so this script comes ready-made. Your Starter project has a `Provided` folder next to `Assets`, with `PoolCounter.cs` in it. Drag the file from your file manager into the `S07_ObjectPooling` folder in Unity's Project window. (It had to wait outside `Assets` because it reads your ObjectPool's `TotalCount` and `ActiveCount`, which didn't exist until section 1.) Double-click it to read along:

- **Three slots:** the two pools and the text to write in.
- **`Update`** adds up how many objects the pools have made (`TotalCount`) and how many are in use (`ActiveCount`).
- **`\n`** starts a new line in the text.
- **Only rebuilding the text when a number changes** matters here: joining strings makes a new string, which is garbage, the very thing the pool avoids.

Add it to **Pool Counter**: drag in **Fireball Pool**, **Impact Pool**, and **Pool Counter** itself into **Counter Text**.

Test it: fire for a while. After hundreds of shots, the pools still hold about 11 fireballs and 6 impacts.

Check your work

- ☐ Holding J fires a stream of fireballs the way you face.
- ☐ Fireballs fly through the fire and potions, and burst on dummies and walls.
- ☐ A dummy takes ten hits.
- ☐ The counter stays small however long you fire.
- ☐ Compare with `CRE132_S07_ObjectPooling_Finished` if something behaves differently.

Key points

- **Pool things that appear and disappear often:** bullets, hit effects, coins, damage numbers, enemies in waves.
- **`OnEnable` is the pooled object's "Start":** it runs every time it comes out of the pool. `Start` runs only once. Reset everything that changes per use.
- **Pick `startSize`** so the pool rarely needs to grow: about the number on screen at once.
- **Unity has a built-in pool too** (`UnityEngine.Pool.ObjectPool<T>`). Writing your own first shows what it does.

If something goes wrong

- **NullReferenceException in PlayerShooter:** the Attack Action, Fireball Pool or Impact Pool slot is empty.
- **NullReferenceException in Projectile's Launch:** the Fireball prefab has no Projectile script, or you added it to a copy in the scene instead of the prefab.
- **Fireballs burst straight away on the player:** check the Player is tagged **Player**.
- **Impacts never disappear:** the Fireball Impact prefab has no ReturnAfterTime.
- **Fireballs vanish as soon as they appear:** `timeLeft = lifetime;` is missing from `Launch`, so the timer has already run out.

Try it

1. Set **Time Between Shots** to 0.03 and watch the pool grow, then raise **Start Size** to match.
2. Pool the coins from S02, so collected coins come back after a few seconds.

3. Change `PlayerShooter` to use `Instantiate` and `Destroy` instead, and compare the **GC Alloc** column in the Profiler (**Window > Analysis > Profiler**).